BrainTumer Project Report

Oct 11, 2026 · @Yatachi

A VGG16 transfer-learning model classifies brain MRI scans into glioma, meningioma, pituitary or no tumor with 91% test accuracy; glioma recall (70%) is the one weak spot, and this report explains every design choice, the results, and how to fix that weakness.

1. Problem statement

Given a single 2-D brain MRI slice, predict which of four classes it belongs to: **glioma**, **meningioma**, **pituitary tumor**, or **no tumor**. This is a multi-class image classification problem.

Why it matters: brain tumors are diagnosed primarily from MRI, and reading scans is slow, specialist-dependent and prone to inter-reader variation. An automated classifier can act as a triage or second-opinion tool: flag likely tumor scans for priority review and suggest the probable type. The three tumor types differ in treatment and prognosis, so distinguishing between them is as important as detecting a tumor at all.

What makes it hard:

- Tumor types can look similar on a single slice; glioma and meningioma in particular overlap in appearance (this shows up in the results).
- MRI scans vary in contrast, brightness, orientation and acquisition protocol.
- Medical datasets are small by deep-learning standards (a few thousand images), so training a CNN from scratch overfits. This is the reason for the transfer-learning approach described later.

Scope: this is an educational / portfolio project, not a clinical tool. Success is measured by accuracy, per-class precision/recall and ROC-AUC on a held-out test set, plus a working web demo.

2. Dataset

Source: the public Brain Tumor MRI Dataset on Kaggle (a merge of the Figshare, SARTAJ and Br35H collections). 7,200 JPEG images, pre-split into training and testing folders, one sub-folder per class.

Class	Training	Testing	Total
glioma	1,400	400	1,800

Class	Training	Testing	Total
meningioma	1,400	400	1,800
notumor	1,400	400	1,800
pituitary	1,400	400	1,800
Total	**5,600**	**1,600**	**7,200**

Three properties of the data shaped the design:

- **Perfectly balanced classes.** Every class has the same count, so plain accuracy is a fair headline metric and no class weighting or oversampling is needed. Per-class recall still matters, because one class can be weak while the average looks fine.
- **Fixed train/test split.** The Kaggle split is used as-is so results are comparable with other work on the same data. The test folder is never used for training; it is evaluated after every epoch only to watch generalisation (no model selection was done on it).
- **Mixed image sizes and formats.** Images come in different resolutions and some are greyscale; everything is resized to 128×128 and converted to 3-channel RGB (see Preprocessing).

Folder layout expected by the code: `data/Training/<class>/*.jpg` and `data/Testing/<class>/*.jpg`. `torchvision.datasets.ImageFolder` reads this directly and assigns labels alphabetically (`glioma=0, meningioma=1, notumor=2, pituitary=3`). The class names are saved inside the model checkpoint so the web app never has to hard-code them — the original Keras reference used `os.listdir` order, which differs between operating systems and is a classic source of silently wrong labels.

3. Project workflow

The pipeline runs end to end in three scripts: `train.py` (data → model → evaluation), `model.py` (shared architecture) and `main.py` (web inference).

1. **Load data** — `ImageFolder` scans `data/Training` and `data/Testing`, infers the four classes from folder names.
2. **Preprocess** — resize to 128×128, convert to RGB, scale pixels to [0, 1]; training images also get random brightness/contrast jitter.
3. **Batch** — `DataLoader` shuffles training data and serves batches of 20 using 2 background worker processes.
4. **Build model** — load VGG16 with ImageNet weights, freeze all convolutional layers except the last three, attach a new 4-class classifier head.

5. **Train** — 5 epochs of Adam (lr 1e-4) on cross-entropy loss; a progress bar shows running loss/accuracy, and test accuracy is reported after each epoch.
6. **Save** — weights plus class names written to `models/model.pt`.
7. **Evaluate** — softmax probabilities on the 1,600 test images → classification report, confusion matrix, one-vs-rest ROC curves with AUC, saved to `outputs/`.
8. **Serve** — `main.py` loads the checkpoint once at startup; a Flask page accepts an upload, applies the same preprocessing, and returns the predicted class with confidence.

Design choice worth noting: the architecture lives in `model.py` and is imported by both `train.py` and `main.py`. The reference project duplicated the preprocessing in two places (notebook and app), which is how train/serve skew creeps in — the app resizing or scaling differently from training would silently degrade predictions.

4. Preprocessing and augmentation

Every step is listed with the reason it is there and what it costs.

Step	What	Why	Trade-off
Resize to 128×128	All images forced to one square size	CNNs need a fixed input; 128 px keeps VGG16 (a heavy network) fast on a laptop GPU and matches the reference notebook	Fine detail is lost; small tumors shrink to a few pixels. 224×224 (VGG16's native size) would likely help accuracy at ~3× compute
Convert to RGB	Greyscale MRIs replicated to 3 channels	VGG16's first layer expects 3 channels because it was pre-trained on colour photos	Triples input memory for no new information; unavoidable with ImageNet weights

Step	What	Why	Trade-off
Scale to [0, 1]	`ToTensor()` divides by 255	Keeps inputs in a small range so gradients are well-behaved; matches the reference	VGG16 was trained on ImageNet mean/std-normalised inputs; not applying that normalisation is a mismatch the fine-tuned layers must absorb (see Next steps)
Brightness jitter ±20%	Random brightness factor in [0.8, 1.2] on training images only	MRI intensity varies between scanners; the model should not learn "brighter = tumor"	Mild; too strong a jitter would wash out tissue contrast
Contrast jitter ±20%	Random contrast factor in [0.8, 1.2]	Same reason; simulates different window/level settings	Same
Shuffle each epoch	`DataLoader(shuffle=True)`	Prevents the network seeing all gliomas, then all meningiomas, which biases the gradient	None

Why augmentation at all. With 5,600 training images a 15-million-parameter network can memorise the set. Augmentation shows the network a slightly different version of each image every epoch, which acts as a regulariser and improves generalisation. The reference notebook used only brightness and contrast, and that was kept for a faithful first port.

What is missing. No geometric augmentation (horizontal flip, small rotation, random crop) was used. For MRI slices a horizontal flip is anatomically valid and small rotations are realistic. This is the cheapest single improvement available, and it is the first recommendation in the Results analysis for lifting glioma recall.

Test images get only the resize and scaling — never augmentation — so the reported metrics reflect real images, not jittered ones. (The reference notebook augmented its test set too, which makes its numbers slightly unreliable.)

5. Model architecture

The model is VGG16 pre-trained on ImageNet, with its classifier removed, the last three convolutional layers fine-tuned, and a new small head for four classes.

```
Input  3 × 128 × 128
|
VGG16 features (13 conv layers, 5 max-pools)      ← ImageNet weights
|   blocks 1–4 (10 conv layers)   frozen
|   block 5   (3 conv layers)     trainable
|
512 × 4 × 4  →  Flatten (8,192)
|
Dropout 0.3
Linear 8,192 → 128, ReLU
Dropout 0.2
Linear 128 → 4   (logits; softmax at inference)
```

Why transfer learning instead of training from scratch. A CNN trained from zero on 5,600 images would need to learn edge, texture and shape detectors from almost nothing and would overfit badly. VGG16's early layers already encode generic visual features learned from 1.2 million ImageNet images; those transfer well even to medical images. Benefit: fast convergence (85% train accuracy after one epoch) and far better generalisation than the data alone could support.

Why VGG16. It was the reference's choice, it is simple (plain 3×3 convs and max-pools, easy to explain and to inspect), and its ImageNet features are known to transfer well. The cost is size: ~15 M parameters in the feature extractor, slow relative to ResNet18 or EfficientNet-B0 which reach similar accuracy at a fraction of the compute. This is flagged under Next steps.

Why freeze blocks 1–4 and train only block 5. Early layers detect edges and textures that are universal; retraining them on a small dataset risks destroying good features and adds nothing. Late layers encode task-specific combinations (what a "tumor-like blob" looks like), so letting block 5 adapt gives the model domain specificity. Freezing also cuts trainable parameters by roughly half and makes each epoch faster. Rule of thumb applied: the smaller the dataset, the fewer layers to unfreeze.

Why remove VGG16's own classifier. The original head outputs 1,000 ImageNet classes and holds ~120 M parameters — far too large for 4 classes and 5,600 images. The replacement head has ~1 M parameters.

Why Dropout (0.3 then 0.2). The 8,192 → 128 linear layer is the single largest trainable block and the easiest place to overfit. Dropout randomly zeroes activations during training so no single feature can be relied on; it is the standard regulariser for dense

layers. Two rates are used because the first layer has far more inputs (8,192) than the second (128) and so needs stronger regularisation.

Why logits rather than a softmax layer. PyTorch's `CrossEntropyLoss` applies log-softmax internally and is numerically more stable than softmax followed by a separate log. Softmax is applied explicitly only when probabilities are needed (ROC curves, the confidence shown in the web app).

Mapping to the original Keras notebook: `include_top=False` → `vgg.features`; `layers[-2], [-3], [-4].trainable = True` → `features[24], [26], [28]`; `Dense(4, softmax)` + `sparse_categorical_crossentropy` → `Linear(128, 4)` + `CrossEntropyLoss`.

6. Training setup

Setting	Value	Why this choice
Loss	Cross-entropy	The standard loss for single-label multi-class problems; it directly maximises the log-probability of the correct class and pairs with softmax outputs. Equivalent to Keras `sparse_categorical_crossentropy` (integer labels, no one-hot needed)
Optimizer	Adam	Adapts a per-parameter learning rate using running averages of gradients and squared gradients, so it converges quickly without hand-tuned schedules. Reliable default for fine-tuning
Learning rate	1e-4	Deliberately small: the pre-trained block-5 weights are already good and a large step (e.g. 1e-3) would overwrite them ("catastrophic forgetting"). Small lr nudges them toward MRI features instead
Trainable params only	`filter(requires_grad)` passed to Adam	Frozen layers are excluded from the optimizer so no memory or time is spent on them

Setting	Value	Why this choice
Batch size	20	Matches the reference; 280 steps per epoch. Small enough to fit comfortably in laptop GPU memory alongside VGG16, and small batches add gradient noise that helps generalisation
Epochs	5	Enough for convergence: train loss falls from 0.39 to 0.04 and test accuracy plateaus by epoch 2 (next section). More epochs mainly increase overfitting without a validation-based stopping rule
Seed	42	`random`, `numpy` and `torch` all seeded so a run can be reproduced
Device	Apple MPS (auto: MPS → CUDA → CPU)	Trains on the Mac GPU; a 5-epoch run takes a few minutes instead of ~an hour on CPU
Data workers	2	Images are decoded and augmented in background processes so the GPU is not left waiting on JPEG decoding

Train / eval mode. `model.train()` is called before each epoch and `model.eval()` before evaluation. This matters here because of Dropout: in training it randomly drops activations, in eval it must be switched off so predictions are deterministic. Forgetting this is a common bug that makes test accuracy look worse than it is.

No gradient at evaluation. Evaluation runs under `torch.no_grad()`, which skips building the autograd graph — roughly halves memory and speeds inference.

What is deliberately not there yet. No validation split, no learning-rate scheduler, no early stopping, no checkpoint-on-best. The test set is observed after each epoch for information only; the final model is simply the epoch-5 weights. These are the standard next additions and are discussed under Next steps.

7. Training progress

The model learned almost everything it would learn in the first two epochs: test accuracy reached ~91% at epoch 2 and stayed flat, while training accuracy kept climbing to ~98%.



Epoch	Train loss	Train accuracy	Test accuracy
1	~0.39	~85%	~88%
2	~0.17	~94%	~91%
3	~0.09	~97%	~90%
4	~0.07	~97%	~92%
5	~0.04	~98%	~91%

Values read from `outputs/training_history.png`; `train.py` prints exact figures per epoch.

What the curves say:

- **Fast convergence** (85% after one epoch) is the transfer-learning payoff: the network started with useful features and only had to adapt block 5 and the head.
- **The gap between train (98%) and test (91%) accuracy is overfitting.** Train loss is still dropping at epoch 5 but test accuracy stopped improving at epoch 2; the extra epochs taught the model the training images, not the task. The ~7-point gap is the headroom that regularisation (more augmentation, early stopping) could recover.
- **Test accuracy wobbles** (91 → 90 → 92 → 91). With no learning-rate decay the optimiser keeps taking the same-sized steps near the optimum. A scheduler such as cosine decay or `ReduceLROnPlateau` would settle it.

- **Epoch 4 was slightly better than epoch 5** on the test set. Because the script saves only the final weights, the shipped model is not the best one seen. Saving on best validation accuracy (with a proper validation split, not the test set) fixes this.

8. Test results

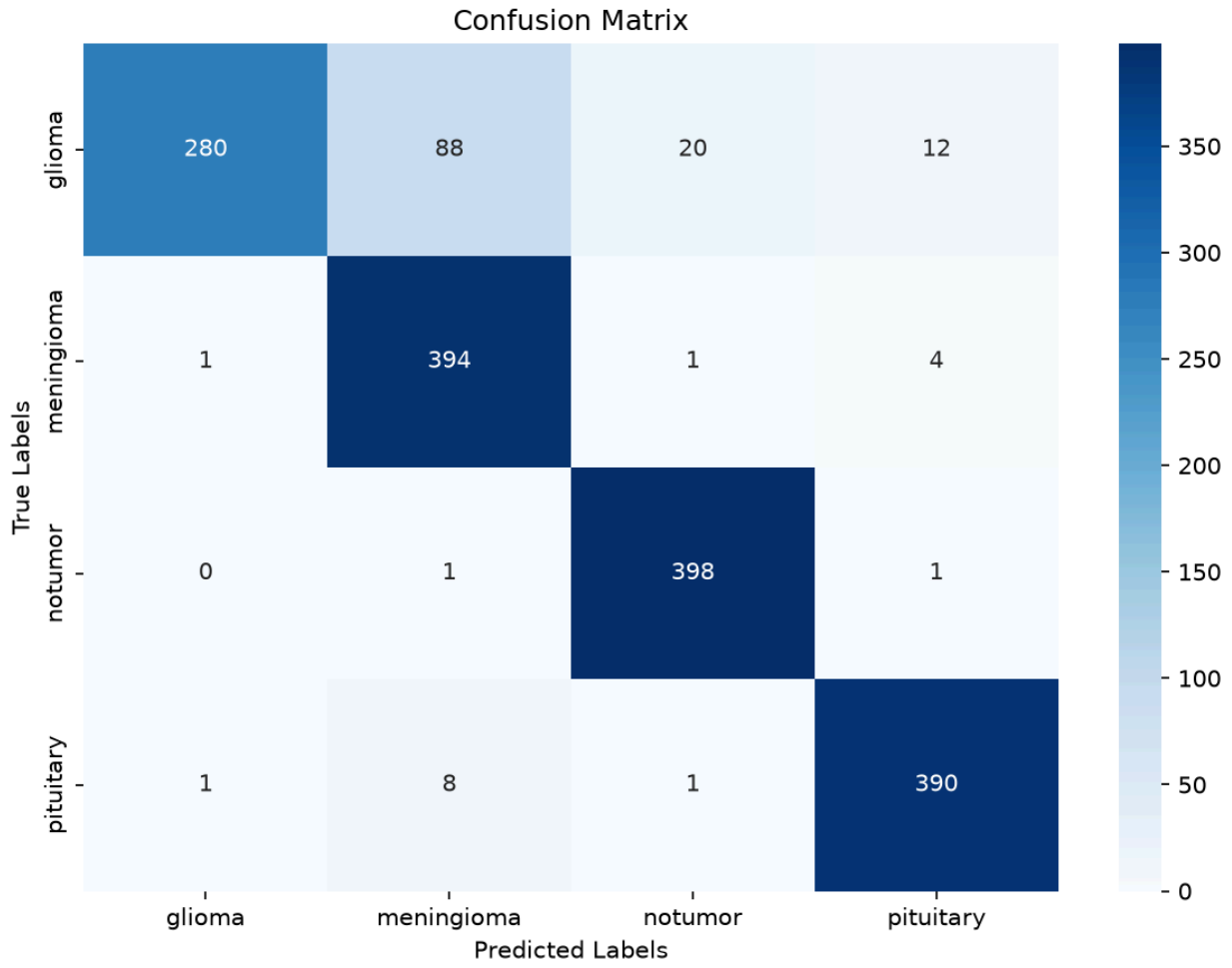
On the 1,600 held-out test images the model scores **91.4% accuracy**, macro F1 0.91, and ROC-AUC between 0.95 and 1.00 per class. Three classes are near-perfect; glioma is the outlier.

Classification report

Class	Precision	Recall	F1	Support
glioma	0.99	**0.70**	0.82	400
meningioma	**0.80**	0.98	0.88	400
notumor	0.95	0.99	0.97	400
pituitary	0.96	0.97	0.97	400
Accuracy			**0.91**	1,600
Macro avg	0.93	0.91	0.91	1,600

How to read it: *precision* = of the images the model called X, how many really were X; *recall* = of the real X images, how many the model found. Glioma has high precision but low recall — when the model says glioma it is almost always right, but it misses 30% of gliomas. Meningioma is the mirror image: it catches 98% of meningiomas but one in five of its "meningioma" calls is wrong, because that is where the missed gliomas go.

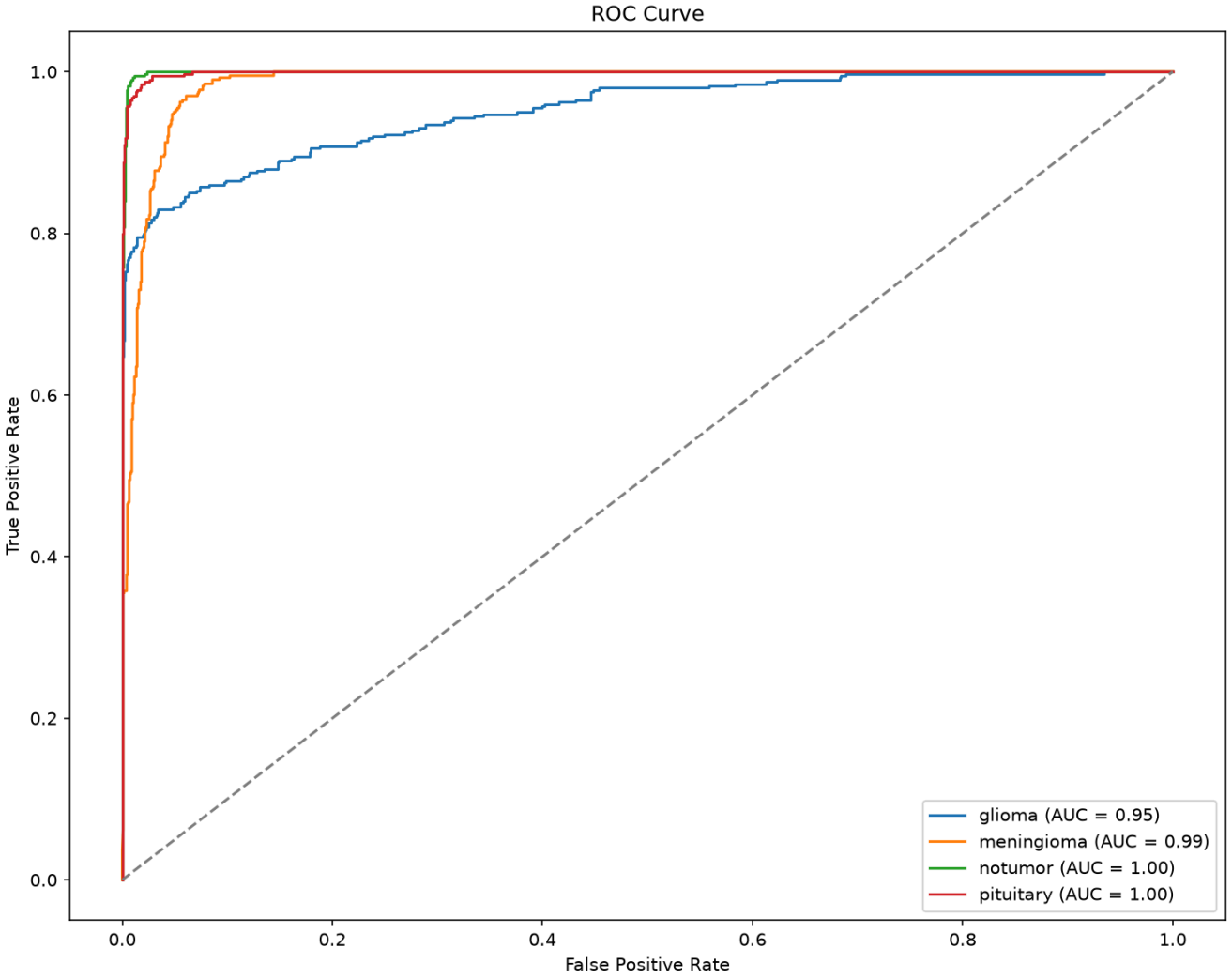
Confusion matrix



True \ Predicted	glioma	meningioma	notumor	pituitary
glioma	**280**	88	20	12
meningioma	1	**394**	1	4
notumor	0	1	**398**	1
pituitary	1	8	1	390

Of the 138 total errors, 120 are on the glioma row: 88 gliomas called meningioma, 20 called "no tumor", 12 called pituitary. The other three classes together account for only 18 mistakes. The 20 glioma → notumor errors are the clinically worst kind (a tumor reported as absent).

ROC curves



Class	AUC
glioma	0.95
meningioma	0.99
notumor	1.00
pituitary	1.00

ROC-AUC measures how well the model's *probability* for a class separates that class from the rest, across every possible threshold — independent of the argmax decision. Glioma's AUC of 0.95 is a useful clue: the model's glioma probabilities rank gliomas above non-gliomas reasonably well, but the fixed argmax rule often picks meningioma anyway. That means the information is partly there and the decision boundary is miscalibrated, which is why threshold adjustment is one of the proposed fixes below.

9. Results analysis

What worked

- **Transfer learning did the heavy lifting.** 91% on a 4-class medical problem after five epochs of a laptop run is only possible because VGG16 arrived with usable features.
- **No-tumor detection is essentially solved** (398/400, AUC 1.00). Healthy brain tissue is visually distinct from any tumor, and the model learned that cleanly.
- **Pituitary is nearly solved** (390/400). Pituitary tumors sit at a fixed, characteristic location (the sella, at the base of the brain), which gives the CNN a strong positional cue.
- **Balanced data kept the model honest** — no class was starved of examples, so the weakness that remains is about appearance, not data quantity.

The problem: glioma recall is 70%

88 of 400 gliomas (22%) are predicted as meningioma, and the AUC of 0.95 confirms glioma is the least separable class. Likely causes, in order of importance:

1. **Visual overlap between glioma and meningioma.** Both can appear as bright, irregular masses with surrounding oedema. Meningiomas are typically well-defined and attached to the meninges, while gliomas are infiltrative and diffuse — but at 128×128 pixels the edge sharpness and boundary texture that distinguish them are largely blurred away. Pituitary and notumor are easy because they differ in *location* or *presence*; glioma vs meningioma differs in *texture*, which is exactly what low resolution destroys.
2. **Asymmetric decision boundary.** The model has effectively learned "if it is a tumor-like mass and not clearly pituitary, say meningioma". Meningioma recall is 98% at the cost of 80% precision — it is absorbing the ambiguous cases. The model is under-confident about glioma rather than ignorant of it.
3. **Glioma is the most heterogeneous class.** Gliomas span grades I–IV and look very different from each other (low-grade ones are subtle, high-grade ones ring-enhancing with necrosis). 1,400 images spread over that variety is thinner coverage than 1,400 images of the comparatively uniform meningioma.
4. **Mild overfitting.** The 98% vs 91% train/test gap means some capacity went into memorising training images. The hardest class suffers most when the model stops generalising.
5. **Missing ImageNet normalisation.** Inputs are scaled to [0,1] but not mean/std-normalised as VGG16 expects. The fine-tuned block 5 compensates, but the frozen blocks 1–4 are seeing slightly off-distribution inputs, which blunts the fine texture features precisely where glioma needs them.

Possible solutions, cheapest first

Fix	What to change	Why it should help glioma	Effort
Geometric augmentation	Add `RandomHorizontalFlip`, `RandomRotation(±10°)`, `RandomResizedCrop(128, scale=(0.85,1))`	More varied gliomas per epoch; reduces overfitting that hits the hardest class first	3 lines
ImageNet normalisation	`Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])` after `ToTensor()` in both train and app	Frozen layers receive the input distribution they were trained on, sharpening texture features	2 lines
Larger input	224×224 (VGG16 native); head becomes 512×7×7 = 25,088 → 128	Restores the boundary and texture detail that separates glioma from meningioma; most likely single biggest gain	1 constant, ~3× slower
Validation split + early stopping + save-best	Hold out ~10% of Training; keep the epoch with best val accuracy	Stops the model after epoch 2–3 where test accuracy peaked, instead of shipping the overfit epoch 5	~30 lines
Learning-rate schedule	`CosineAnnealingLR` or `ReduceLROnPlateau`	Removes the epoch-to-epoch wobble; lets the model settle into a better minimum	3 lines
Class-weighted loss or decision threshold	Weight glioma higher in `CrossEntropyLoss`, or at inference scale glioma's probability up before argmax	Directly shifts the glioma/meningioma boundary; trades some meningioma recall for glioma recall. AUC 0.95 shows the probabilities already carry the signal	1–5 lines
Unfreeze block 4 as well	Train `features[17..23]` too, with lr 1e-5 for the backbone and 1e-4 for the head	More capacity to learn MRI-specific mid-level texture; needs the lower lr and augmentation to avoid overfitting	5 lines

Fix	What to change	Why it should help glioma	Effort
Longer training with the above	10–15 epochs once regularisation and early stopping exist	Only worthwhile after the fixes above; alone it just overfits more	—
Different backbone	ResNet50 or EfficientNet-B0 at 224×224	Deeper features, batch-norm, far fewer parameters than VGG16's head; often +2–4 points on this dataset in published comparisons	Swap `model.py`
Test-time augmentation	Average predictions over the image and its horizontal flip	Cheap ~0.5–1 point gain, smooths borderline glioma/meningioma calls	5 lines in `main.py`

A realistic target after the first four fixes is 95–97% accuracy with glioma recall above 90%, which is where published VGG16/ResNet results on this dataset sit.

Which metric to optimise

For a triage tool, **recall on tumor classes** matters more than precision: a missed tumor is worse than an unnecessary specialist review. The 20 glioma → notumor errors should therefore be weighted most heavily in any further tuning, even though they are a small fraction of the total. A simple safeguard in the app: if the "notumor" probability is below a high threshold (e.g. 0.9), show "inconclusive — review recommended" instead of "No Tumor".

10. Web application

`main.py` is a small Flask app that turns the checkpoint into an interactive demo: upload an MRI image, get the predicted class and a confidence percentage, see the image back.

How a request flows:

1. On startup the app calls `load_checkpoint('models/model.pt')` **once**, rebuilds `TumorClassifier`, loads the weights, switches to `eval()` mode and moves to the best device. Loading per request would add seconds of latency.

2. A `POST /` with a file saves it under `uploads/` (filename sanitised with `secure_filename` so a crafted name cannot escape the folder).
3. The image is opened with Pillow, converted to RGB, resized to 128×128 and scaled to [0, 1] — the **same transform as the test set**, imported from the same `model.py` constants, so training and serving cannot drift apart.
4. A forward pass under `torch.no_grad()` returns logits; softmax turns them into probabilities; the argmax is the predicted class and its probability is the confidence shown.
5. `notumor` is rendered as "No Tumor", any other class as "Tumor: <type>"; the template shows the result card with the uploaded image.

Why Flask: it is minimal, the reference used it, and a single-page upload form does not need more. The template is served from `templates/index.html` (Flask's required location; the reference had it at the project root, which would have raised `TemplateNotFound`).

Caveat for users of the demo: the confidence is the softmax probability, which is **not** a calibrated probability of being correct. Neural networks trained with cross-entropy are typically over-confident; a 95% glioma call does not mean 95 in 100 such images are glioma. Temperature scaling on a validation set would calibrate it.

11. Limitations and next steps

Limitations

- **Single 2-D slice.** Radiologists read a full 3-D volume across several sequences (T1, T2, FLAIR, contrast). One slice carries far less information, so this model's ceiling is below clinical practice.
- **Test set used for monitoring.** Test accuracy was printed per epoch. No decision was made on it, but strictly a separate validation split should carry that role; the reported 91% is a fair estimate but not a sealed one.
- **Dataset provenance.** The Kaggle set merges three sources with different scanners and labelling standards, and some images in the glioma/meningioma folders are known to be mislabelled in the original SARTAJ subset. Part of the glioma confusion may be label noise, not model error.
- **Resolution.** 128×128 discards detail; the model never sees what distinguishes a glioma edge from a meningioma edge.
- **No explainability.** The app says *what* but not *where*. Without a saliency map a user cannot check whether the model looked at the tumor or at an artefact.

- **Not a medical device.** No regulatory validation, no external test cohort, no clinician review.

Next steps, in recommended order

- [ ] Add geometric augmentation and ImageNet normalisation; retrain (expected: glioma recall ↑, train/test gap ↓)
- [ ] Carve a 10% validation split from Training; add early stopping and save-best checkpointing
- [ ] Move to 224×224 input and compare against 128×128 on the same split
- [ ] Add a cosine learning-rate schedule and train 10–15 epochs
- [ ] Try ResNet50 / EfficientNet-B0 backbones in `model.py` and compare accuracy vs inference time
- [ ] Tune the glioma decision threshold or class weight so tumor recall is prioritised
- [ ] Add Grad-CAM heat-maps to the web app so each prediction shows where the model looked
- [ ] Calibrate confidence with temperature scaling and show an "inconclusive" state below a threshold
- [ ] Log every run (settings + metrics) so experiments are comparable

Reproducing this report

```
python train.py            # 5 epochs, batch 20, lr 1e-4, seed 42
python main.py             # http://127.0.0.1:5000
```

Outputs: `models/model.pt`, `outputs/classification_report.txt`, `outputs/training_history.png`, `outputs/confusion_matrix.png`, `outputs/roc_curve.png`.

Sources: Brain Tumor MRI Dataset, Kaggle · VGG16 — Simonyan & Zisserman, 2014 · torchvision VGG16 weights